Embedded Data Structure
(Lecture Note 3)

Revisit C Programming (2)

2025. 3. 11

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Basic concept of array and pointer**
  - **Array and pointer are same except for …**
  - **Remember usage of array and pointer**
  - **Remember the definition of pointer**

    : Pointer is a local variable containing the address of the variable

※ Please draw the procedure related to pointers !

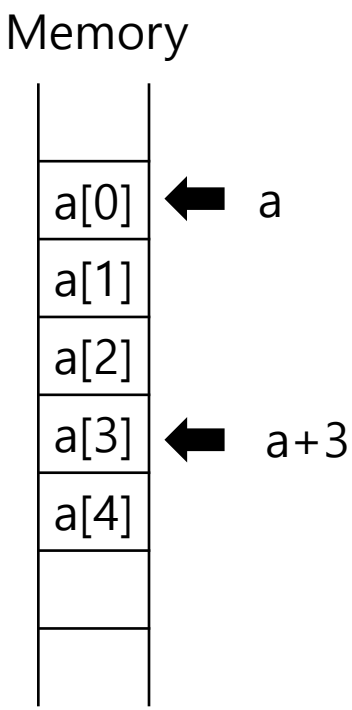
Pointer and Array (1/2)

- **The name of array is "pointer"**
  - **Both of them are the same except for "static" vs. "dynamic"**

→ Indicate the same memory space with two different ways

```c
#include <stdio.h>

void main()
{
    int a[] = {1, 2, 3, 4, 5};

    printf("%d₩n", &a[0]);
    printf("%d₩n", &a[1]);
    printf("%d₩n", &a[2]);
    printf("%d₩n", a);
}
```

```c
#include <stdio.h>

void main()
{
    int a[] = {1, 2, 3, 4, 5};

    printf("%d₩n", a);
    printf("%d₩n", a+1);
    printf("%d₩n", *a);
    printf("%d₩n", *(a+1));
}
```



➔ &a[i] = ? / a[i] = ? (think about the difference)

Pointer and Array (2/2)

- **The name of array is "pointer" – cont'd**
  - **You need to access memory with freely pointers**

→ Indicate the same memory space with two different ways

```
#include <stdio.h>

void main()
{
    int a[] = {1, 2, 3, 4, 5};
    int *p;

    p = a;
    printf("%d  %d  %d₩n", a[1], a[2], a[3]);
    printf("%d  %d  %d₩n", p[1], p[2], p[3]);

}
```

What happens exactly ?

Draw the procedure for this !

```
p[1] = 7;
p[2] = 8;
p[3] = 9;
```

Pointer and Function (1/4)

- **Pointer also can be employed as a parameter**
  - **We provide the "address" (not the "value") for functions**

→ Formulation is the same as the function prototype

```
void main()
{
    ...
    compute_sum(grade, int N);
    ...
}
```

[main.cpp]

```
void compute_sum(int *score, int N)
{
    ...
    sum += score[j];
    ...
}
```

[compute.cpp]

※ To do this, grade should be defined as a pointer, e.g., int *grade;

(Can you explain the procedure by arrows ?)

※ Once again, pointers can be directly represented using the array

KU KONKUK UNIVERSITY

▪ Call-by-value vs. Call-by-reference

- **Call-by-value : copy value (function inside)**

※ Example : change two variables using a swap function defined as

```
#include <stdio.h>

void swap(int x, int y);

void main()
{
    int a = 100, b = 200;
    printf("a = %d,  b = %d₩n", a, b);
    swap(a, b);
    printf("a = %d,  b = %d₩n", a, b);
}
```

a b (copy)

```
void swap(int x, int y)
{
    int temp;

    temp = x;
    x = y;
    y = temp;
}
```

[swap function]

➡ **What is the final result ? Is it changed ?**

KU KONKUK UNIVERSITY

▪ Call-by-value vs. Call-by-reference – cont'd

- **Call-by-reference : copy address !**

※ Example : change two variables using a swap function defined as

```
#include <stdio.h>

void swap(int x, int y);

void main()
{
   int a = 100, b = 200;
   printf("a = %d,  b = %d₩n", a, b);
   swap(  ,  );
   printf("a = %d,  b = %d₩n", a, b);
}
```

```
void swap(int *x, int *y)
{
   int temp;

   temp = x;
   x = y;
   y = temp;
}
```

Modify it !

[swap function]

➡ **What is the final result ? Is it changed ?**

Pointer and Function (4/4)

KU KONKUK UNIVERSITY

▪ Some examples

- **Try to apply pointers to your functions !**
  - Fill out blanks (understand the pointer operations)

```
#include <stdio.h>

int *add(int x, int y);

void main()
{
    int *result;
    result = add(5, 9);
}

int *add(int x, int y)
{
    int *z;
    *z = x + y;
    return [      ]
}
```

```
#include <stdio.h>

void sub(int x[], int n);

void main()
{
    int a[3] = {1, 2, 3};

    sub(a, 3);

    printf("%d %d %d₩n",
            a[0], a[1], a[2]);
}
```

```
void sub(int x[], int n)
{
    for(int j = 0; j<n; j++)
        x[j] -= 1;
}
```

Array

```
void sub([      ], int n)
{
    for(int j = 0; j<n; j++)
        [      ] -= 1;
}
```

Pointer

Function Pointer

▪ Function name as a pointer

- **Function pointer : indicating the "code" not the "data"**

```
#include <stdio.h>

int function_A(int N)
{
    int a;
    for(a = 0; N > 0; N--)
        a += N;
    return a;
}
```

What does this function do ?

➔ Function pointer : int (*fptr) (int);

※ Bracket is required

※ Usage : type (pointer) (type)

Consider following sentences :

1) fptr = function_A

2) printf("%d₩n", function_A(100));
   printf("%d₩n", fptr(100));

This will be frequently employed with void pointer

➔ Make a generalized version

Void Pointer (1/2)

- **Void : no type (empty)**
  - **It does not indicate the data type → Why do we use it ?**

```
void *vptr;

int a;
a = *vptr;
a =*(int *)vptr;

a = *(vptr + 1);
a =
```

Key advantage of void pointers :

It is useful for making the generalized function

(i.e., it works regardless of data types)

Fill out the blank using pointer casting

* Void pointer cannot use the dereference, **why ?**

* Void pointer cannot use the pointer operators, **why ?**

➔ **Type casting** should be employed !

Void Pointer (2/2)

- **Application : function generalization**
  - **Make complete the generalized version of swap function**

```
#include <stdio.h>
#include <stdlib.h>

void swap(void *a, void *b, int n)
{
    void *t;
    t = malloc(n);
    memcpy(t, a, n);
    memcpy(a, b, n);
    memcpy(b, t, n);
    free(t);
}
```

➔ **Why using "n" ?**

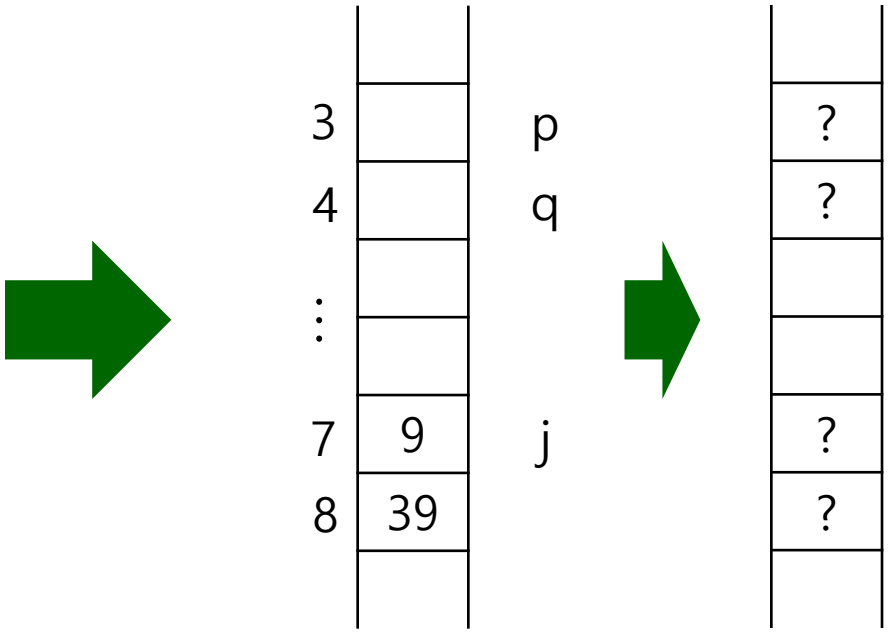
```
void main()
{
    char c1 = 'A', c2 = 'B';
    int i1 = 3, i2 = 9;
    float f1 = 3.3, f2 = 9.9;

    // for char type
    swap(                    );

    // for int type
    swap(                    );

    // for float type
    swap(                    );
}
```

Pointer of Pointer (1/4)

- **Pointer of pointer is also an address**
  - **Once again, interpret clearly the following statement :**

```
int **q;
```

```
#include <stdio.h>

void main()
{
    int j = 9;
    int *p = &j;
    int **q = &p;

    *p = 10;
    **q = 9;
}
```



Pointer of Pointer (2/4)

- **Let's consider the cartesian coordinate (or matrix)**
  - **Example : think about scanning a given image**

int S[3][5];

S[0][0]	S[0][1]	S[0][2]	S[0][3]	S[0][4]
S[1][0]	S[1][1]	S[1][2]	S[1][3]	S[1][4]
S[2][0]	S[2][1]	S[2][2]	S[2][3]	S[2][4]

How are such arrays assigned in the memory space (physically) ?

S[0][0]	S[0][1]	S[0][2]	S[0][3]	S[0][4]	S[1][0]	…	S[2][0]	S[2][1]	S[2][2]	S[2][3]	S[2][4]

1st row 3rd row

This is because the structure of the memory is …

Pointer of Pointer (3/4)

KU KONKUK UNIVERSITY

▪ Initialization of the two-dimensional array

- **Consider the following example :**

```
int array[3][3] =
{  {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}  };
```

```
int array[][3] =
{  {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}  };
```

⇨ **Which one is correct ? Why ?**

What is array[2][1], array[1][1] ?

Starting address ➔ what is this ?

…	1	2	3	4	5	6	7	8	9	0	…

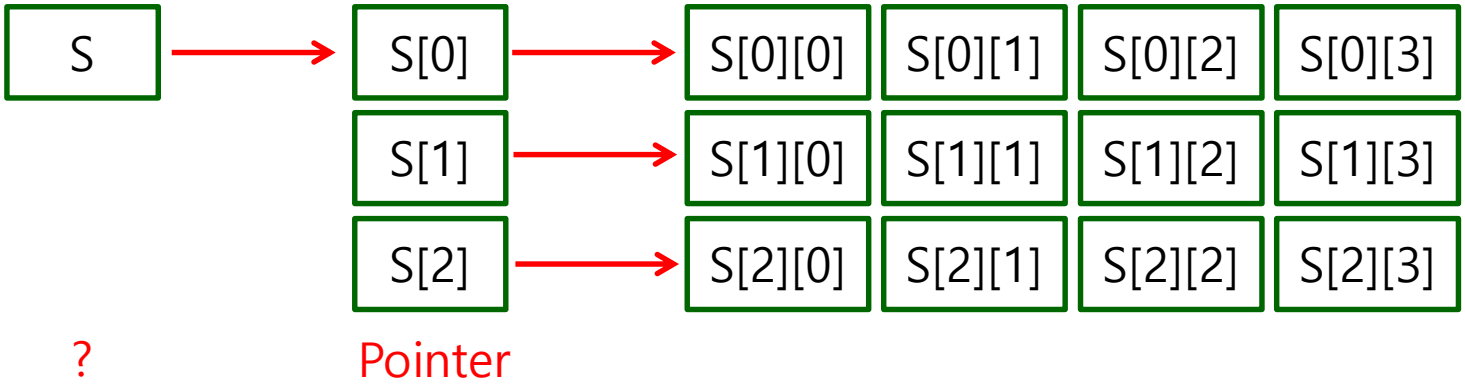
*** Name "array" indicates three (int types) pointers in the memory space !**

➔ Where do **array+1** and **array+2** indicate in the memory ?

* Think about the relationship with the pointer of pointer

Pointer of Pointer (4/4)

- **2-dimensional array**
  - **Note : memory structure is still linear (this indexing is just for us !)**

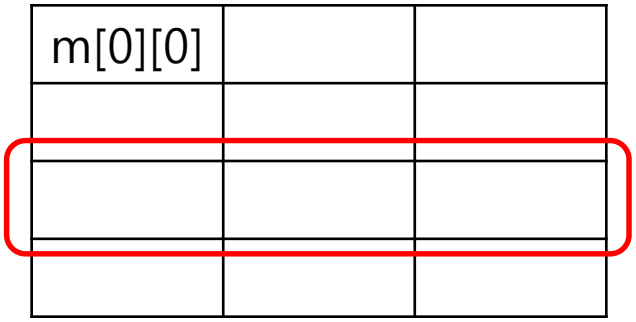



```
#include <stdio.h>

void main()
{
   int a[2][2] = { {1,2}, {3,4} };

   printf("%d₩n", [          ]); // print out a value at (1,0) (or (1,1))
}
```

Fill out the blank only using a pointer !

Practice Example

- **Compute the mean of a specific row (or column)**
  - **Why ? memory management in the embedded system**




```
#define ROWS    4
#define COLS    3

int m[ROWS][COLS] = {{1, 3, 5},
{9, 7, 3}, {3, 3, 9}, {2, 8, 6}};
```

```
float get_row_avg(int m[][COLS], int r){
    int *start, *end;
    float avg = 0;

    start = [          ];
    end = [          ];

    while(start <= end){
        avg += [          ];
        [          ];
    }

    avg /= COLS;

    return avg;
}
```

Summary

- **Basic concept of advanced operations for pointers**
  - **Relationship between array and pointer**
  - **Pointer and function / void pointer (casting)**
  - **Function pointer / pointer of pointer**

    ※ Remember the definition of "pointer of pointer"